

Capitolo 4

Metamodello della documentazione e regole di authoring DDTA

Stato del capitolo

Questo capitolo descrive la baseline di ricerca raggiunta al commit DDTA 9a04bc5b98b7f4a2b8400f2b210e7cea6b04d1a9. Il perimetro è intenzionalmente limitato alla scrittura e al metamodello della documentazione fino al *Functional Requirement*. Il modello di *Security Requirement*, la Base Analysis, i Base Analysis Elements (BAE), gli overlay di analisi e i plugin metodologici sono fasi successive. La baseline qui descritta è quindi congelata per procedere con la ricerca, ma resta riapribile in presenza di un controesempio concreto prodotto dalle fasi successive.

4.1 Scopo del capitolo

La Documentation-Driven Threat Analysis (DDTA) assume che la documentazione governata di un progetto non sia soltanto un supporto narrativo per persone già a conoscenza del sistema, ma possa costituire una sorgente di conoscenza sufficientemente stabile, strutturata e tracciabile da alimentare analisi successive. Questo obiettivo richiede tuttavia un passaggio preliminare: prima di definire un modello analitico, occorre stabilire *come* la conoscenza progettuale debba essere scritta, organizzata e progressivamente specializzata.

Il problema affrontato in questo capitolo è pertanto anteriore al threat modeling vero e proprio. Non si chiede ancora quali minacce debbano essere identificate, né quale metodologia debba essere applicata. Si chiede invece quale struttura documentale consenta a stakeholder con competenze differenti di descrivere in modo governato:

- il problema generale affrontato dal progetto;

- le macro-responsabilità necessarie a trattarlo;
- le scelte che restringono lo spazio delle soluzioni;
- i comportamenti operativi che devono risultare da tali scelte;
- le identità e le relazioni che dovranno essere riusate nelle fasi successive senza duplicazione semantica.

La risposta proposta è un metamodello documentale progressivo, nel quale il percorso principale è normalizzato come:

```
Project problem framing      [method precondition]
  -> Macro Requirement
  -> Decision
  -> Functional Requirement
  -> Specialized Requirement
```

Per il perimetro della tesi, la prima specializzazione che verrà formalizzata dopo questo capitolo è il *Security Requirement*. Tale specializzazione verrà definita prima di applicare una metodologia come STRIDE, così da non rendere la semantica del requisito di sicurezza dipendente dalla tassonomia o dal formato di output di un particolare metodo di analisi.

4.2 Separazione tra semantica, rappresentazione e tooling

Un rischio emerso durante la costruzione del corpus è confondere il modello concettuale con la sua serializzazione corrente o con le funzionalità di uno strumento. DDTA separa quindi quattro livelli.

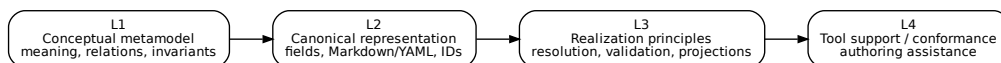


Figura 4.1: Separazione tra metamodello, rappresentazione, principi di realizzazione e supporto del tool.

L1 - metamodello concettuale. Definisce significato, concetti, relazioni, cardinalità, nullabilità semantica e invarianti. Una proprietà appartiene a L1 soltanto se deve rimanere vera indipendentemente dal formato dei file e dal software utilizzato.

L2 - rappresentazione canonica. Stabilisce come la semantica L1 viene materializzata: campi YAML, sezioni Markdown, registry, convenzioni di identificazione, ordering, mirror e altri dettagli Doc-as-Code.

L3 - principi di realizzazione del modello. Comprende risoluzione di riferimenti, validazione, indici, proiezioni, suggerimenti di authoring e altre funzioni che rendono il modello eseguibile o utilizzabile su larga scala.

L4 - supporto e conformità del tool. Descrive quali contratti DDTA una specifica implementazione, come ThreatForge, supporta, valida o assiste.

Questa separazione è particolarmente importante per la tesi. Una regola non diventa parte del metamodello soltanto perché ThreatForge la implementa; analogamente, un campo storico di ThreatForge non diventa automaticamente un requisito del modello generale. Il criterio utilizzato è un test di sostituibilità: se lo strumento venisse sostituito, la proprietà continuerebbe a descrivere il progetto governato? In caso affermativo è candidata a L1/L2; in caso negativo appartiene più probabilmente a L3/L4 o all'architettura del tool.

4.3 Problem framing come preconditione metodologica

4.3.1 Perché il modello non parte direttamente dagli MR

I primi esperimenti hanno mostrato che un *Macro Requirement* può diventare accidentalmente una descrizione della soluzione corrente. Un progetto di controllo degli accessi, ad esempio, può essere formulato sin dall'inizio come “sistema di riconoscimento facciale con telecamera e modello ML”. Tale formulazione è leggibile, ma contiene già decisioni: il controllo potrebbe essere realizzato tramite badge, impronta, volto, combinazioni di fattori o un servizio esterno. Se queste scelte entrano nell'MR, la successiva gerarchia documentale non rende più visibile dove e perché lo spazio delle soluzioni è stato ristretto.

Per ridurre questo rischio, DDTA introduce un passo di *problem framing* prima della decomposizione in documenti governati. Il framing non è, nella baseline corrente, un nuovo tipo di documento canonico; è una preconditione del metodo di authoring.

La domanda fondamentale è:

Quale problema generale o classe di problemi deve affrontare il progetto, entro quale confine significativo, se rimuoviamo temporaneamente le scelte di soluzione correnti?

Il framing deve utilizzare vocabolario del problema e del dominio. Nomi di provider, database, algoritmi, framework, device o metodologie di threat analysis devono essere rimossi a meno che siano intrinseci al problema stesso. La stessa operazione viene usata come test di stabilità: se la formulazione smette di avere senso quando si sostituisce una tecnologia, è probabile che una Decision sia stata incorporata troppo in alto.

4.3.2 Funzioni del framing

Il framing svolge quattro funzioni metodologiche. In primo luogo stabilisce un confine condiviso prima che il progetto venga scomposto. In secondo luogo permette di controllare se gli MR coprono il problema senza sovrapporsi eccessivamente. In terzo luogo crea un riferimento stabile per riconoscere la soluzione-vocabulary leakage. Infine supporta la teachability: un nuovo lettore può comprendere prima il problema e soltanto successivamente le scelte progettuali.

Un esempio sintetico è il seguente:

```
Problema:
controllare l'accesso fisico a un'area protetta

Non assunto nel framing:
- riconoscimento facciale;
- impronta digitale;
- telecamera;
- modello ML;
- database locale;
- identity provider esterno.
```

Il framing non crea quindi un livello documentale aggiuntivo. Evita piuttosto che il primo livello governato contenga già informazioni che dovrebbero emergere come scelte.

4.4 Gerarchia documentale progressiva

La baseline della tesi adotta una gerarchia intenzionalmente regolare.

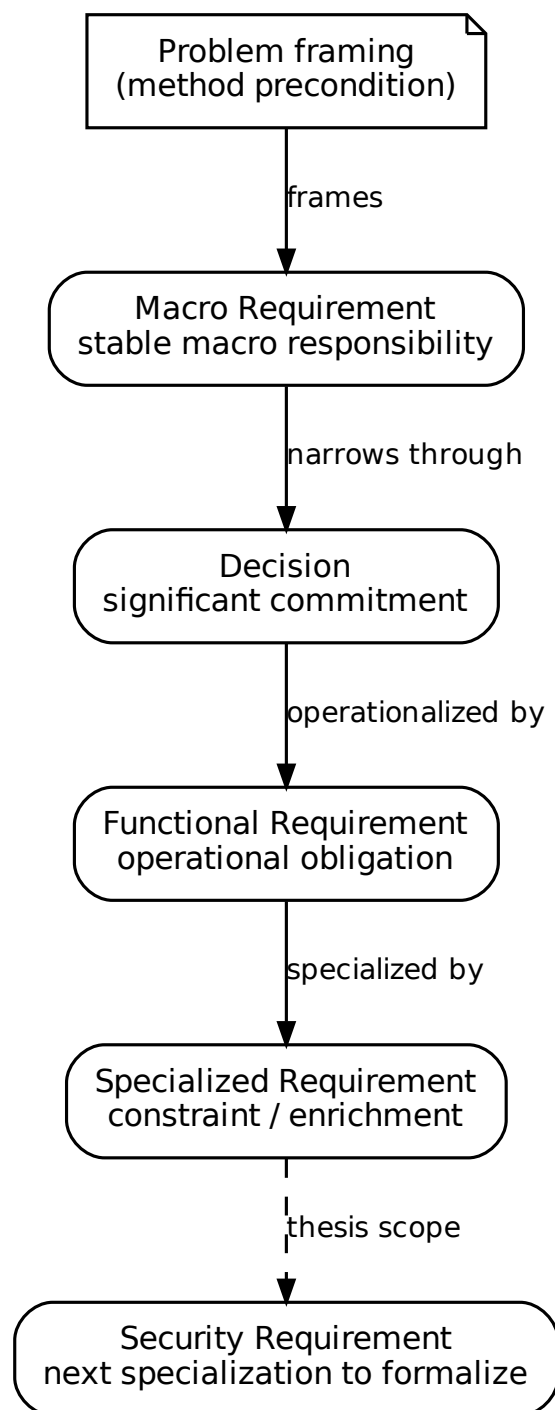


Figura 4.2: Gerarchia documentale progressiva della baseline DDTA.

La gerarchia rappresenta *contenimento semantico*, non il parsing di un identificatore o di una cartella. Un *Macro Requirement* possiede una famiglia di Decision; una Decision restringe quell'MR mediante un commitment governato; un *Functional Requirement* rende operativo il commitment; un requisito specializzato aggiunge un vincolo verticale alla funzione senza sostituirla.

Le identità testuali possono in futuro riflettere la gerarchia per ragioni di usabilità, ma non devono esserne la fonte di verità. Una relazione parentale deve essere semanticamente esplicita e validabile anche se cambia il formato degli ID.

4.5 Macro Requirement

4.5.1 Definizione

Un *Macro Requirement* è il documento radice che possiede una responsabilità macro durevole necessaria a trattare il problema definito dal framing. Spiega il valore e il risultato macro perseguito, il contesto minimo, gli stakeholder e il confine della responsabilità, senza selezionare una soluzione lower-level e senza specificare comportamenti atomici indipendentemente verificabili.

La domanda fondamentale è:

Nel problema di progetto già inquadrato, quale responsabilità o concern macro stabile stiamo governando, quale risultato o valore di livello macro deve contribuire, perché conta e chi coinvolge?

Questa definizione separa l'MR da due concetti adiacenti. Se una frase esprime una scelta che potrebbe essere sostituita con un'altra posizione, appartiene normalmente a Decision. Se esprime un comportamento operativo che può essere valutato indipendentemente, appartiene normalmente a *Functional Requirement*.

4.5.2 Forma semantica

La baseline richiede i concetti riportati nella Tabella 4.1.

Tabella 4.1: Forma semantica del Macro Requirement.

Concetto	Stato	Funzione
id	richiesto	identità stabile e tracciabilità
title	richiesto	nome conciso comprensibile nel dominio
lifecycle	richiesto	stato di governance corrente/storico
intent	richiesto	scopo, valore e risultato macro desiderato

Concetto	Stato	Funzione
context	richiesto	background minimo necessario all'interpretazione
stakeholders	richiesto	ruoli beneficiari, coinvolti, governanti o impattati
scope	richiesto se-manticamente	confine della responsabilità macro
assumptions	opzionale	fatti trattati come veri per l'intero branch
constraints	opzionale	limiti applicabili all'intero branch
dependsOn	0..* MR	dipendenze macro non gerarchiche

Un campo separato **Objective** non è mantenuto nel core: nei corpus esaminati duplicava l'Intent oppure iniziava a decomporlo in comportamenti lower-level. L'Intent deve essere conciso, ma sufficientemente informativo da permettere a uno stakeholder competente nel dominio di comprendere il risultato macro cercato.

4.5.3 Invarianti dell'MR

Gli invarianti chiusi per la baseline documentale sono i seguenti:

1. **Root hierarchy:** l'MR non ha parent gerarchico.
2. **Stable identity:** l'identità non dipende dalla forma testuale dell'ID.
3. **Problem anchoring:** l'MR deve essere interpretabile nel problem framing del progetto.
4. **Single macro concern:** un MR possiede una sola responsabilità macro coerente.
5. **Child narrowing:** Decision e FR discendenti restringono la responsabilità, non introducono scope estraneo.
6. **No solution commitment leakage:** provider, architetture, algoritmi o meccanismi scelti appartengono a Decision, salvo che siano parte intrinseca del dominio del problema.
7. **No FR-level obligation:** comportamenti atomici e operativi appartengono downstream.
8. **Broad domain readability:** il documento è leggibile da stakeholder competenti nel dominio senza richiedere conoscenza dell'implementazione corrente.

9. **Architecture resilience**: sostituire una soluzione lower-level non dovrebbe normalmente richiedere la riscrittura dell'MR.
10. **Temporal stability**: una nuova Decision non rende obsoleto l'MR se la responsabilità macro resta invariata.
11. **Explicit dependency**: relazioni complementari tra MR non devono essere nascoste come pseudo-gerarchia.
12. **Teachability**: il set di title e Intent degli MR deve sostenere una mappa macro comprensibile del progetto.
13. **Analysis enabling, not analysis producing**: l'MR può fornire contesto utile, ma non crea automaticamente elementi analitici o categorie di metodo.
14. **Projection honesty**: una mappa didattica degli MR non deve essere presentata come DFD o threat model canonico.

4.5.4 Split, merge e dipendenze tra MR

La decomposizione non segue automaticamente l'architettura software. Due sottosistemi non richiedono necessariamente due MR, e due responsabilità di dominio non devono essere fuse soltanto perché oggi sono implementate dallo stesso componente. Il test principale è la stabilità della responsabilità dopo la rimozione del vocabolario della soluzione.

Quando due MR collaborano per fornire uno scopo end-to-end, la collaborazione non implica co-ownership. Un MR può dipendere da una capability prodotta da un altro; la dipendenza macro deve essere resa esplicita senza duplicare la responsabilità. Questo principio verrà raffinato nella Base Analysis, dove dovrà essere definito l'oggetto canonico del consumo di servizio.

4.6 Decision

4.6.1 Dalla “scelta” al commitment significativo

Una *Decision* è un commitment significativo che restringe esattamente un *Macro Requirement*. Il termine commitment è preferito a una definizione basata soltanto sulla presenza di alternative, perché alcune decisioni sono determinate da vincoli regolatori, contrattuali, fisici o organizzativi. Ciò che conta è che il documento registri conoscenza non derivabile che cambia il responsibility boundary, la strategia, la policy, l'architettura o un altro aspetto significativo del progetto e produca conseguenze downstream.

La domanda fondamentale è:

Quale commitment significativo restringe questo Macro Requirement, perché il progetto assume questa posizione e quali conseguenze accettiamo?

La forma minima rimane volutamente piccola:

```
Decision
|- title
|- context
|- decision
'- consequences
```

Tutti e quattro i concetti sono richiesti e non nulli. **Context** è locale alla **Decision**: descrive la tensione, il problema irrisolto o il vincolo che rende necessario prendere posizione. **Decision** contiene una sola posizione coerente. **Consequences** rende visibili gli effetti materiali e i downstream obligations. **Alternative** e **Rationale** possono essere utili nella rappresentazione, ma non sono campi L1 obbligatori distinti.

4.6.2 Decision di responsibility boundary

Uno dei risultati più importanti dei probe è la necessità di rendere esplicito *chi* possiede un comportamento. Una capability può essere implementata dal progetto oppure consumata da un sistema esterno. Questa scelta modifica direttamente il tipo di FR che il progetto deve possedere.

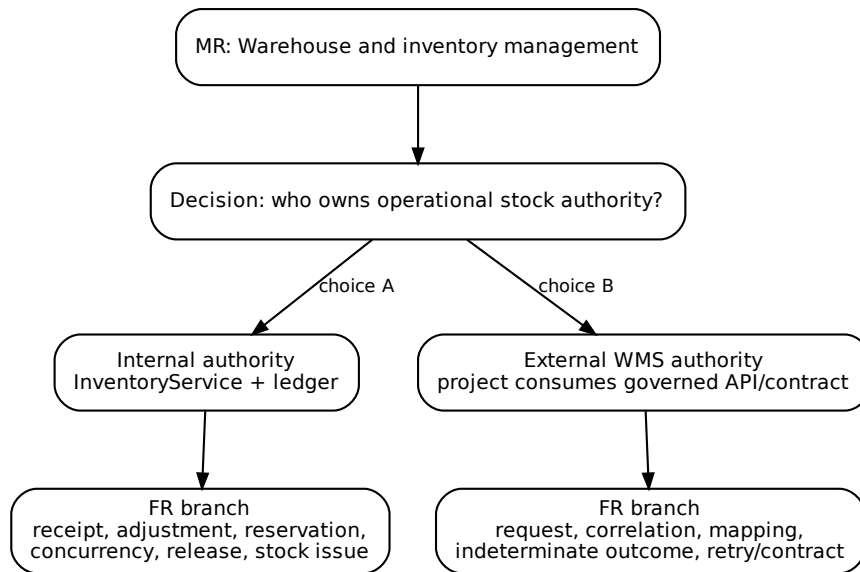


Figura 4.3: Una Decision sul responsibility boundary modifica il branch di FR downstream.

Nel caso del magazzino, scegliere una authority interna produce obblighi relativi a carico, rettifica, reservation, concorrenza, release e stock issue. Scegliere un WMS esterno sostituisce gran parte di tali obblighi con FR di integrazione: richiesta al provider, correlazione, mapping dei risultati, gestione degli outcome indeterminati, retry e conformità al contratto di interfaccia. Gli invarianti interni del WMS non diventano automaticamente FR del progetto.

Questa osservazione ha corretto i primi esempi, nei quali alcuni FR sembravano discendere direttamente dall'MR soltanto perché la Decision di responsibility boundary era stata assunta implicitamente.

4.6.3 Necessity/default Decision

La baseline della tesi privilegia una gerarchia costante MR -> Decision -> FR. In un caso raro può non emergere una alternativa materialmente distinta. Rimuovere il layer Decision semplificherebbe localmente la semantica, ma renderebbe variabile il modello, gli strumenti di navigazione e le regole di validazione. DDTA adotta quindi una normalizzazione controllata: la *necessity/default Decision*.

Questa Decision è valida soltanto se:

1. il Context spiega perché alternative materialmente diverse sono assenti, non ammissibili o escluse da un vincolo governato;
2. il commitment non è una tautologia del tipo “il sistema farà ciò che dice l’MR”;
3. le Consequences sono materiali e creano responsabilità o vincoli downstream;
4. il documento può essere riaperto o superseded se il vincolo cambia o appare una nuova alternativa;
5. la formula non viene usata per mascherare la mancata esplorazione delle alternative.

La necessity/default Decision non afferma quindi che esistano alternative inesistenti. Documenta, al contrario, che il progetto ha esaminato il punto di commitment e che lo spazio ammissibile è risultato singleton o non materialmente differenziato.

4.6.4 Split e non ridondanza

Una Decision deve essere divisa quando parti del commitment possono cambiare indipendentemente e possiedono conseguenze o trade-off autonomi. Per esempio, scegliere il riconoscimento facciale come modalità di verifica e scegliere dove risiedono le reference biometriche sono decisioni separabili: il provider delle reference può cambiare lasciando invariata la modalità di verifica.

Viceversa, non si deve creare una Decision per ogni frase tecnica. Una specifica esprime una Decision soltanto quando governa una scelta o un commitment significativo. Meccanismi incidentali, tuning, nomi di classi, path di sorgente o dettagli che possono cambiare senza modificare il contratto governato appartengono alla realizzazione.

4.7 Functional Requirement

4.7.1 Definizione operativa

Un *Functional Requirement* è un obbligo operativo governato e indipendentemente valutabile che discende da esattamente una Decision. Esprime un comportamento, un servizio, una transizione di stato o un risultato osservabile che il progetto deve fornire nel contesto stabilito dal branch MR/Decision.

La domanda fondamentale è:

Data la Decision parent e una condizione o input applicabile, quale soggetto o capability governata deve compiere quale azione funzionale su o con quale oggetto/informazione, e quale risultato osservabile o comportamento di failure deve conseguire?

Il vincolo “operativo” è essenziale. Frasi come “fornire disponibilità di magazzino” o “mantenere l’autorizzazione aggiornata” sono spesso troppo deboli: non chiariscono lifecycle, correlation, concurrency, failure semantics o boundary di responsabilità. Un FR deve specificare abbastanza comportamento da consentire a un implementatore di capire cosa deve accadere e a un reviewer di stabilire se l’obbligo è soddisfatto, senza imporre dettagli incidentali di implementazione.

4.7.2 Gerarchia e ownership

La baseline chiude le seguenti regole:

- ogni FR ha esattamente una Decision parent;
- l’MR owner è derivato dal parent della Decision e non costituisce una seconda autorità topologica;
- nessuna Decision è figlia di FR;
- un FR può possedere zero o più requisiti specializzati;
- un nuovo commitment scoperto durante l’implementazione o la review deve essere registrato come Decision sotto l’MR appropriato, con revisione o supersession degli FR interessati.

La gerarchia costante è una scelta metodologica di questa tesi. I primi controesempi a un parent Decision obbligatorio sono stati ritirati come evidenza dopo che una decomposizione più realistica ha mostrato Decision nascoste. Il modello resta comunque riapribile qualora una fase successiva dimostri un fallimento ricorrente non risolvibile mediante la necessity/default Decision.

4.7.3 Forma semantica

La forma concettuale corrente è:

```
FunctionalRequirement
|- id / lifecycle
|- title
|- functionalObligation
|- functionalClause 1..*
|   |- normativeStatement
|   '- spoReference 1..*
|       |- subject
|       |- predicate
|       '- object
```

```
|- parentDecision -> Decision 1
'- specializedRequirement -> 0..*
```

Non è richiesto un Intent locale all'FR: la ragione per cui la funzione esiste dovrebbe essere recuperabile dall'MR e dalla Decision. L'FR possiede il *cosa deve funzionare*. Acceptance criteria, prove di soddisfazione e artifact di implementazione rimangono relazioni downstream e non vengono confusi con la semantica del requisito.

4.7.4 Normative prose e SPO

I probe hanno mostrato che una notazione puramente relazionale è insufficiente a descrivere obblighi reali. Concorrenza, idempotenza, condizioni, correlazione e failure semantics non possono essere ridotte senza perdita a una singola tripla. Per questo la prosa normativa rimane primaria, mentre la forma Subject-Predicate-Object (SPO) rende esplicite le identità e le azioni rilevanti.

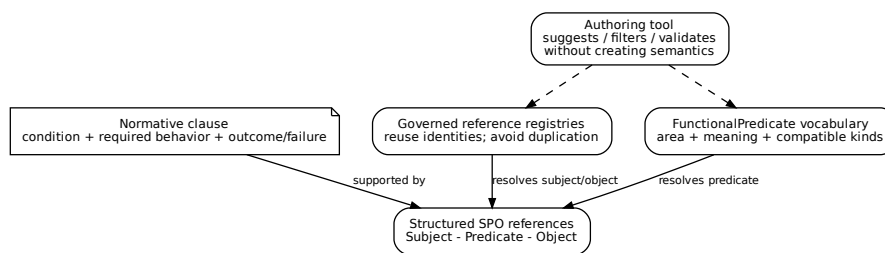


Figura 4.4: Relazione tra clausola normativa, riferimenti SPO, registri governati e supporto del tool.

Un pattern leggibile è:

```
[Condition,] <Subject> MUST <Predicate> <Object>
[qualifier / observable outcome / failure behavior].
```

Un esempio tratto dal corpus di magazzino è:

```
Quando una ReservationRequest identifica l'OrderEvaluation attiva e l'intero
    set
richiesto, InventoryService MUST creare una Reservation ACTIVE soltanto se
    il set
completo puo' essere allocato atomicamente sullo stock corrente; una
    richiesta
```

```
concorrente MUST NOT riservare quantita' gia' congelata dalla Reservation
attiva.
```

Le relazioni strutturate associate possono essere:

```
InventoryService -- creates --> Reservation
InventoryService -- updates --> InventoryBalance
InventoryService -- produces --> ReservationResult
```

Le triple non sostituiscono la clausola: aiutano a riusare identità governate, costruire indici, validare coerenza e preparare l'integrazione con la Base Analysis.

4.7.5 FunctionalPredicate

Il Predicate non è una stringa libera diversa per ogni documento. DDTA prevede un metacontratto governato per un *FunctionalPredicate*, comprendente almeno:

- identità stabile;
- label leggibile;
- significato normativo;
- area funzionale;
- tipi compatibili di subject e object, da formalizzare dopo la chiusura della tassonomia Base Analysis.

Il core non congela un elenco esaustivo di verbi. Sono invece possibili vocabolari estendibili per area: storage (read, write, update, delete), communication (send, receive, request), lifecycle (activate, suspend, release, expire, purge), cryptography (encrypt, decrypt, hash), signature (sign, verify signature), identity/access e altre aree di dominio.

Questa impostazione rende possibile un supporto di authoring in cui il tool propone subject e object già presenti nel registro e filtra i Predicate compatibili. La proposta non crea però semantica canonica: l'autore o il processo di review rimangono l'autorità che conferma l'identità e il significato del riferimento.

4.7.6 Regola sui parametri

Il corpus ha mostrato che valori numerici o modalità operative cambiano per ragioni semanticamente differenti. Inserire un campo generico `functionalParameters` nell'FR mescolerebbe responsabilità diverse. Il metodo richiede invece una classificazione.

- “singolo frame vs N frame vs breve video” descrive una strategia e appartiene normalmente a Decision;
- un target di latenza o throughput è candidato a requisito di performance/quality;
- una durata di retention è candidata a governance/privacy/policy;
- un limite imposto da un’API può appartenere a un InterfaceContract;
- buffer, thread count o tuning incidentale appartengono alla realizzazione/configurazione.

L’FR deve referenziare la regola governata applicabile quando il valore è necessario, evitando di copiare parametri mutevoli in più documenti.

4.7.7 Composizione tra MR senza doppia ownership

Un servizio end-to-end nasce spesso dalla collaborazione di più branch. Il branch ordine può consumare il servizio di reservation del branch magazzino e l’autorizzazione del branch pagamento. Questo non crea due owner per lo stesso FR.

La semantica chiusa è:

Il consumo di una capability governata da un altro MR non trasferisce ownership e non crea un secondo parent per l’FR consumer.

Non è ancora chiuso, invece, il target canonico della relazione di consumo. Potrebbe essere un FR provider, una capability, un service, un BAE o un InterfaceContract. La scelta viene deliberatamente rimandata alla Base Analysis, per non forzare una relazione FR -> FR prima di avere un modello strutturale del sistema.

4.8 Regole di authoring end-to-end

La gerarchia non è soltanto una classificazione. Deve guidare il processo di scrittura. La sequenza proposta è:

1. **Frame the problem.** Definire problema generale, boundary e out-of-scope rimuovendo temporaneamente la solution vocabulary.
2. **Decompose into MR.** Identificare responsabilità macro stabili, controllando overlap, dipendenze e resilienza alle scelte future.

3. **Expose Decisions.** Per ogni branch chiedere quali commitment definiscono responsibility boundary, policy, strategia, architettura o contratto significativo. Se il solution space è realmente singleton, documentare una necessity/default Decision.
4. **Write operational FRs.** Tradurre ogni Decision in comportamenti osservabili e indipendentemente valutabili.
5. **Add structured references.** Collegare la prosa normativa a subject, predicate e object governati senza duplicare le identità.
6. **Review cross-branch consumption.** Rendere visibili le capability consumate da altri MR senza trasferire ownership.
7. **Specialize downward.** Aggiungere requisiti verticali come SecurityRequirement senza riscrivere il comportamento funzionale di base.

Per ogni documento viene usata una domanda fondamentale differente. Questo è importante per evitare documenti formalmente corretti ma semanticamente collocati nel layer sbagliato:

Tabella 4.2: Domande di authoring per i livelli documentali.

Livello	Domanda
Problem framing	Quale problema o classe di problemi affrontiamo se rimuoviamo le soluzioni correnti?
MR	Quale responsabilità macro stabile governiamo, quale valore produce e chi coinvolge?
Decision	Quale commitment significativo restringe questa responsabilità e quali conseguenze accettiamo?
FR	Dato quel commitment, quale comportamento operativo deve accadere, su quali soggetti/oggetti, con quale risultato o failure behavior?
Specialized Requirement	Quale vincolo verticale aggiuntivo deve rispettare la funzione senza sostituirla la semantica?

4.9 Ruolo del tool nell'authoring

La regolarità della gerarchia è stata scelta anche perché consente un'assistenza più semplice e prevedibile. Questa motivazione non è però sufficiente da sola a definire la semantica: il tool segue il metamodello, non lo determina.

Un sistema come ThreatForge può assistere l'autore in almeno cinque modi:

- mostrare la gerarchia e impedire parentage non ammessi;
- suggerire riferimenti già esistenti, evitando duplicazione di subject/object;
- filtrare i Predicate compatibili con l'area e con i tipi governati;
- segnalare incoerenze tra prosa normativa e SPO;
- avviare una proposta esplicita quando manca una voce nel registro, mantenendo separata la proposta dalla canonizzazione.

Non deve invece estrarre automaticamente nomi dalla prosa e trasformarli in elementi canonici senza review, né usare similarity ranking per decidere equivalenza, parentage o ownership. Search e suggerimenti sono strumenti di scalabilità; l'autorità semantica resta governata.

4.10 Validazione mediante probe iterativi

4.10.1 Perché gli esempi iniziali non sono stati trattati come prova definitiva

La costruzione del modello ha seguito una sequenza di esempi, controesempi e regressioni. Due famiglie di errori sono state particolarmente informative.

Nel dominio del controllo accessi, un primo corpus assumeva già “riconoscimento facciale”, “telecamera” e “ML” a livello MR. Alcuni FR sembravano quindi non richiedere una Decision. Rimuovendo tali assunzioni è emerso che la modalità di verifica, la strategia di acquisizione e la source delle reference erano commitment distinti. Il controesempio al parent Decision obbligatorio risultava contaminato dalla presenza di Decision nascoste nei livelli superiori.

Nel primo holdout order/inventory, la funzione di magazzino era ridotta a una query di disponibilità. L'esempio ignorava carico, scarico, rettifiche, reservation lifecycle e concorrenza; questa semplificazione rendeva plausibile un FR diretto dall'MR soltanto perché non era stato modellato il responsibility boundary reale.

Questi esempi non vengono usati come evidenza di closure. Sono mantenuti come storia del processo di falsificazione: mostrano perché un corpus realistico deve essere abbastanza dettagliato da esporre le scelte che un modello troppo sintetico può nascondere.

4.10.2 Probe order fulfillment e warehouse management R3

Il corpus R3 descrive un problema order-to-fulfillment con stock fisico finito, checkout concorrenti, movimenti di magazzino, reservation temporanee, payment authorization, fulfillment e dispatch. Il framing non assume WMS/ERP locale o esterno, database, provider, carrier, protocollo o algoritmo.

Il branch di magazzino introduce esplicitamente una Decision sulla authority operativa: il probe sceglie gestione interna tramite InventoryService e InventoryLedger, ma documenta come alternativa un WMS/ERP esterno. Questa Decision cambia materialmente gli FR downstream e dimostra l'importanza di non confondere “il sistema deve governare il magazzino” con “il progetto deve implementare internamente il ledger”.

Altre Decision distinguono on-hand/reserved/blocked, scelgono reservation temporanee, definiscono atomicità all-lines e stabiliscono milestone di fulfillment. Gli FR risultanti specificano idempotenza, correlazione, concurrency, release/expiry e stock issue senza prescrivere lock, database isolation level o framework.

Il risultato rilevante non è che ogni dominio debba avere queste stesse Decision. Il corpus dimostra piuttosto che una descrizione operativa sufficientemente ricca porta le scelte di responsibility e strategy a diventare visibili prima degli FR, sostenendo la gerarchia regolare adottata per il perimetro della tesi.

4.11 Confine verso Security Requirement e Base Analysis

La chiusura fino a FR non equivale alla chiusura dell'intero DDTA. Il documento produce un input governato, ma non stabilisce ancora come tutti i subject, object, service e interaction debbano essere rappresentati in un grafo analitico comune.

La fase immediatamente successiva verrà divisa intenzionalmente in due passi. Prima verrà definito il metamodello method-neutral di *Security Requirement*. Successivamente verranno formalizzati Base Analysis e BAE, quindi il contratto generico di analisi e overlay. Soltanto dopo verrà applicato STRIDE come primo metodo concreto.

Questa sequenza evita una contaminazione importante: se il SecurityRequirement venisse progettato dopo STRIDE, campi o relazioni potrebbero essere introdotti soltanto perché convenienti per categorie come Spoofing o Tampering. Definendo prima il requisito di sicurezza, il test diventa più forte: STRIDE dovrà produrre Finding che possano condurre a istanze del modello SEC già stabilito, non il contrario.

Documentation baseline

MR -> Decision -> FR

```

    |
    v
SpecializedRequirement common semantics
    |
    v
SecurityRequirement metamodel          [method-neutral]
    |
    v
Base Analysis / BAE
    |
    v
Analysis Record / Finding common envelope
    |
    v
method-neutral overlay/plugin contract
    |
    v
STRIDE as first concrete method
    |
    v
accepted Finding -> SecurityRequirement instance/provenance
    |
    v
effective obligation -> design change -> re-analysis

```

Elementi intenzionalmente non chiusi in questo capitolo

Restano fuori dal perimetro: tassonomia BAE, semantica di DataStore/Channel/Interface-Contract, target canonico della service consumption, cardinalità e provenance del Security-Requirement, Analysis Record, Finding, overlay/plugin, STRIDE, STRIDE-AI e meccanismi di stale/change impact. Questi concetti non devono essere anticipati retroattivamente dentro MR/Decision/FR salvo che un controesempio concreto dimostri l'insufficienza della baseline.

4.12 Condizioni di riapertura del metamodello documentale

Il modello descritto in questo capitolo è un checkpoint di ricerca, non una dichiarazione di completezza universale. Per evitare modifiche opportunistiche, vengono stabilite condizioni di riapertura esplicite.

MR, Decision o FR devono essere riaperti soltanto se una fase successiva mostra almeno uno dei seguenti problemi:

1. un SecurityRequirement method-neutral non può specializzare correttamente un FR senza duplicarne o deformarne la semantica;
2. la Base Analysis non può essere costruita o integrata a partire dalla conoscenza governata senza richiedere informazioni che il modello di authoring impedisce di esprimere;
3. un overlay metodologico richiede source information che non può essere ottenuta senza contaminare la documentazione di progetto con categorie method-specific;
4. il layer Decision costante produce un errore semantico ricorrente e concreto, non soltanto il costo occasionale di una necessity/default Decision;
5. la service composition richiede ownership multipla o una relazione gerarchica incompatibile con il modello corrente.

La correzione, se necessaria, dovrà essere attribuita a un controesempio osservabile e seguita da regressione sugli esempi già accettati.

4.13 Conclusioni

La baseline documentale DDTA separa tre domande progettuali che nella documentazione ordinaria tendono spesso a mescolarsi. L'MR stabilisce *quale responsabilità macro* appartiene al progetto; la Decision stabilisce *quale commitment* restringe tale responsabilità; l'FR stabilisce *quale comportamento operativo* deve conseguire. La specializzazione aggiunge vincoli verticali senza invertire la direzione della gerarchia.

Il problem framing riduce l'anticipazione della soluzione; la Decision di responsibility boundary rende visibile ciò che il progetto implementa e ciò che consuma; la necessity/default Decision conserva una gerarchia costante nei rari casi singleton; l'FR combina prosa normativa e SPO per mantenere leggibilità e riferimenti strutturati; il tool può assistere authoring e riuso senza diventare autorità semantica.

Questo capitolo chiude quindi, per il perimetro corrente della tesi, il problema di *come scrivere la documentazione governata fino al Functional Requirement*. Il prossimo passo non è ancora STRIDE. Verrà prima definito il SecurityRequirement come specializzazione metodologicamente neutra, poi il modello Base Analysis/BAE e soltanto successivamente il contratto di overlay e la prima applicazione concreta di una metodologia di threat analysis.